

4. Inheritance

What

- same thing like **inheritance** in life
- Child inherits attributes and behaviors from the parent class

When

- when we find some relation and we are able to answer the question
 - **is a relation?**

Need

CONTEXT :- I'm recreating this universe. I'm at the stage where i need to define life in my world. So here is how i will do it (Keep in mind :- I'm a lazy person)

```
[STEP-I] :- Defining what is meaning of life
  - anything with that can show some kind of loco movements
  - if it reproduces || respirates || eats || dies
  - it will be called a life
//base thing
{
    bool reproduces;
    bool eats;
    bool respriates;
    bool movements;
    list how it grows;
}
[STEP-II] :- defining the other details
                                   (in sort :- specializations)
- animal{
    - range of food
    - how it eats
    - level
    - legs
    - color
    - is Predator
} + `the things i defined earlier`

- human {
    - skin color
```

```

- veg or non veg
- age
- testicals or not
- sleeps or not
} + *animal* "things" + `the things i defined earlier`

```

// reason why i'll like to have some way to copy my code(without actually copying it)

Mode of inheritance

	Derived Class	Derived Class	Derived Class
Base Class	Private	Public	Public
Private	Not Inherited	Not Inherited	Not Inherited
Protected	Private	Protected	Protected
Public	Private	Protected	Public

Types

```

A
|
B

```

```

A
|
B
|
C

```

```

  A
 / \
B   C

```

```

  A   B
   \ /
   C

```

```

  A
 / \
B   C

```



Advantages

- Re-usability
- Less code => Less point of errors
- Extensibility
- Data Hiding (Encapsulated by default)

Basic Syntax

```
// three interpretations
//1
class DerivedClass : accessModifier BaseClass{};
//2
class sonClass : accessModifier dadClass{};
//3
class pankaj : accessModifier Medhansh{};
```

Access Modifiers

Private Public

Protected

```
class B{
    public:
        int a;
        int b;
};

class A : public B{};
```

```
A
|
B

class B: public A{
    //Some code here
};
```

A
|
B
|
C

```
class B: public A{  
    //Some code here  
};  
  
class C: public B{  
    //Some code here  
};
```

A
/ \
B C

```
class B: public A{  
    //Some code here  
};  
class C: public A{  
    //Some code here  
};
```

A B
 \ /
 C

```
class B: public A ,public B{  
    //Some code here  
};
```

A
/ \
B C
 \ /
 D

```
class B: public A{  
    //Some code here  
};
```

```

class C: public A {
    //Some code here
};

class D: public B ,public C{
    //Some code here
};

```

Constructor

- The main goal of the OOPs is to decrease repetitive code writing so.
 - We don't actually need to rewrite the constructor of the derived class over again
- Here's an example

```

class A{
    //some attributes
public:
    A(int a,int b,int c){
        // setting these attributes
    }
};

class C{
    //some attributes
public:
    A(int d,int e,int f){
        // setting these attributes
    }
};

class B : public A{
    //some more attributes
public:
    B(int a,int b,int c,int d,int e,int f,int g) :
        A(a,b,c),
        C(d,e,f)
    {
        this->g = g;
    }
};

```

NOTE

- inheritance should always form a DAG (Direct acyclic graph)

CONST vs CONST vs CONST

L-value and R-value

type	properties	examples
L-value	Variable having memory locations	<code>int a = 10</code>
R-value	Variable not having memory locations	<code>5</code> , <code>int &a = b</code> //something like alias

Const Variable

- Const variable

```
const int p = 19;  
p = 10; ❌ → error
```

- Const pointer cases
- *Pointer to const data*

```
const int *p = &x; //cannot change *p  
int const *p = &x; //Same as above  
*p = 10; ❌ → error
```

- *Const pointer*
 - Like the java-script one the address is fixed but not the actual data

```
int* const p = &x; // cannot change p  
*p = 10; ✅  
  
// one example  
int* const arr = new int[10];  
arr[0] = 19; // valid
```

- *Const pointer to const data*

```
const int* const p = &x; // cannot change *p && // cannot change p
```

 Note

const before * makes the pointer data constant
To make the location constant we use it after the *

- **Const function arguments**
- *Prevents modification inside function*

```
void func(const int a) {  
    // a cannot be changed  
}
```

- *With references (important)*

```
void func(const int& a) {  
    // avoids copy + protects value  
}
```

- **Const return type**
- Prevents modification of returned value (mostly useful with references/pointers)

```
const int& getVal();
```

- **Const member function**
- *Guarantees object state won't change*

```
class A {  
public:  
    int x;  
    int get() const {  
        return x; // cannot modify x here  
    }  
    // *Guarantees object state won't change*  
};
```

- **Const objects**
 - Can only call const member functions

```
const A obj;  
obj.get();    // ✓  
// obj.set(); // ✗ if not const
```

- **Const with arrays / STL**

```
void print(const vector<int>& v) {  
    // cannot modify v  
}
```

```
}
```

Upcasting

- The parent class can hold the child class

```
class A{
    //somethings
};

class B : public A{
    //something too
};

int main(){
    B classB;
    A classA = &classB;
}
```

mutable keyword

- mutable allows a class member to be modified even inside a const object or const member function
- only used for debugging purposes

```
class A {
public:
    mutable int x;

    void show() const {
        x = 10;
    }
};
```

- cannot be used the local variables
- and function arguments
- it is exception to the **const** keyword